Wiki
SQL: http://en.wikipedia.org/wiki/SQL
SQL syntax: https://en.wikipedia.org/wiki/SQL_syntax

basic example

```
SELECT 1;

 ?column?
══════════
        1
(1 row)

SELECT 1 AS x, 'a' AS name UNION SELECT 2, 'b';

 x | name
═══╪══════
 1 | a
 2 | b
```

*** Basic syntax**
[SQL 2023] 6.3 <value expression primary>

```
-- comment
/*
 multi
 line comment
 */
mytable          -- name e.g. table name
MyTable          -- BAD. name is case insensitive by
default
my table         -- BAD
"MyTable"        -- GOOD
"my table"       -- GOOD
'my table'       -- GOTCHA: '' is a string
"my column"      -- e.g. column name
```

*** NULL**

**NULL**
**NULL** represents an **unknown** value.
Thus, every **operation** with NULL is also NULL.

**GOTCHA**: NULL comparison is also **always NULL**, including self.
Use IS NULL or DISTINCT FROM

```
-- comparison
x = NULL         -- NULL
NULL = NULL      -- NULL
x <> NULL        -- NULL
x < NULL         -- NULL
```

```
NULL < NULL      -- NULL
x <= NULL        -- NULL
x > NULL         -- NULL
x >= NULL

-- OK
NULL IS NULL                  -- T
1 IS NULL                     -- F
1 IS DISTINCT FROM NULL       -- T
NULL IS DISTINCT FROM NULL    -- F
```

```
-- e.g. CAST: NULL CAST is always NULL
CAST(NULL AS INT)         -- NULL
CAST(NULL AS FLOAT)       -- NULL
CAST(NULL AS mytype)      -- NULL
...
```

```
-- e.g. predicate: NULL comparisons are NULL
1 = NULL                      -- NULL
1 <> NULL
1 < NULL
NULL AND TRUE                 -- NULL
NULL AND FALSE                -- NULL
-- self comparisons are also NULL
NULL = NULL                   -- NULL
NULL <> NULL                  -- NULL
NULL IS NULL                  -- TRUE
NULL IS DISTINCT FROM NULL    -- F
```

```
-- e.g. expression: all NULL operations are NULL
1 + NULL         -- NULL
1 / NULL         -- NULL
```

***) usage: NULL with GROUP BY and DISTINCT**

NULL values are **NOT comparable** in normal expressions
i.e. NULL = NULL  -- NULL

In DISTINCT and GROUP BY, NULL values are treated as one group

```
-- GROUP BY
SELECT x
FROM (VALUES (1), (NULL), (NULL), (2)) _(x)
GROUP BY x;

  x
------
 NULL
    1
```

```
   2
(3 rows)


-- DISTINCT
SELECT DISTINCT x
FROM (VALUES (1), (NULL), (NULL), (2)) _(x) ;

  x
______
 NULL
    1
    2
(3 rows)
```

==========

## == Data types ==

Wiki: https://en.wikipedia.org/wiki/SQL_syntax#Data_types
[SQL 2023] 4.2 Data types
[SQL 2023] 6.1 <data type>

*** Boolean**
[SQL 2023] 4.6 Boolean
[SQL 2023] 6.1 <data type>

```
-- Boolean
BOOLEAN           TRUE/FALSE
```

*** Numeric**
[SQL 2023] 4.5 Numbers
[SQL 2023] 6.1 <data type>

**Numeric type**
Numerics are either:

a) **exact numeric type**: represents a number within a given **absolute range**.
Any number outside the range cannot be represented.
Decimal points are **rounded**.
e.g.

```
CAST(1 AS SMALLINT)       -- OK
CAST(1e+20 AS SMALLINT)   -- ERROR
```

b) **approximate numeric type**: represents an approximation.
i.e. number stored upto a **precision** aka **significant digits**.
e.g.

```
CAST(1 AS REAL)               -- OK
CAST(123456789 AS REAL)       -- OK 1.2345679e+8
```

```
CAST(0.123456789 AS REAL)     -- OK 0.12345679
```

**Precision**
**Scale**
**Precision** is the number of **significant digits**
**Scale** is **lowest digits below decimal**
Thus, the value must be

| | |
|---|---|
| **10^(p-s) > value > -10^(p-s)** | p = **precision**<br>s = **scale** |

e.g.

| | **Form** | **Range** |
|---|---|---|
| (5, 2)<br>**precision 5**<br>**scale 2** | XXX.YY | (-1000, 1000)<br>**min**: 0.01<br>**max**: 999.99 |
| (2, 4)<br>**precision 2**<br>**scale 4** | 0.00YY | (-0.01, 0.01)<br>**min**: 0.0001<br>**max**: 0.0099 |

e.g. exact type

```
-- XXX.YY
CAST(1 AS NUMERIC(5,2))           -- 1
CAST(123.45 AS NUMERIC(5,2))      -- 123.45
CAST(999.99 AS NUMERIC(5,2))      -- MAX: 999.99
CAST(1000 AS NUMERIC(5,2))        -- ERROR above max
CAST(0.01 AS NUMERIC(5,2))        -- MIN: 0.01
CAST(0.005 AS NUMERIC(5,2))       -- 0.01 rounded up to
XXX.YY
CAST(0.004 AS NUMERIC(5,2))       -- 0 rounded down to
XXX.YY

-- 0.00YY
CAST(0.0012 AS NUMERIC(2, 4))     -- 0.0012
CAST(0.0099 AS NUMERIC(2, 4))     -- MAX: 0.0099
CAST(0.01 AS NUMERIC(2, 4))       -- ERROR above max
CAST(0.00994 AS NUMERIC(2, 4))    -- 0.0099 rounded down
to 0.00YY
CAST(0.00995 AS NUMERIC(2, 4))    -- ERROR rounded up to
0.00YY
CAST(0.00005 AS NUMERIC(2, 4))    -- 0.0001 rounded up
to 0.00YY
CAST(0.00004 AS NUMERIC(2, 4))    -- 0 rounded up to
0.00YY
```

e.g. approximate type: can only specify precision

```
-- max 8 significant digits
CAST (12.34 AS REAL);              -- 12.34
CAST (1.23456789 AS REAL);         -- 1.2345679 rounded
```

```
up to X.YYYYYYY
CAST (123456789 AS REAL);          -- 1.2345679+08
rounded up to X.YYYYYY
CAST (123456789.1234 AS REAL);     -- 1.2345679+08


---- Exact types
--
-- a) integral exact type
-- Exact precision depends on implementation
SMALLINT                  small integer
                          e.g. 2 bytes  [-32768, 32767]
INTEGER                   medium integer
                          e.g. 4 bytes  [-2147483648,
2147483647]
INT                       same
BIGINT                    large integer
                          e.g. 8 bytes
[-9223372036854775807, 9223372036854775808]
NUMERIC(4)


-- b) general exact type
-- Exact precision depends on implementation
NUMERIC                   arbitrary i.e. any number
NUMERIC(5)                precision 5, scale 0
                          i.e. 5 digit integer
NUMERIC(5,2)              precision 5, scale 2
                          i.e. XXX.YY >= value >= -XXX.YY
NUMERIC(1, 3)             0.00Y >= value >= -0.00Y
NUMERIC(0)                ERROR
DECIMAL(5,2)              ?
DECLOAT(5,2)              ?


-- b) approximate type
-- Exact precision depends on implementation
FLOAT
FLOAT(4)                  custom precision
REAL                      small precision
                          e.g. 6 decimal precision
DOUBLE PRECISION          large precision
                          e.g. 12 decimal precision

-- exact precision
NUMERIC                   arbitrary precision
DECIMAL                   same
DEC
NUMERIC(5, 2)             precision=5  scale = 2
NUMERIC(5)
DECFLOAT(5)               ?
DECFLOAT
```

```
-- NUMERIC(5,2)
CAST (12 AS NUMERIC(5,2))              12.000
CAST (12345 AS NUMERIC(5,2))           ERROR. Overflow
CAST (12.1234567 AS NUMERIC(5,2))      12.346

-- NUMERIC i.e. arbitrary
CAST (123456789123456789.23456789123456789 AS NUMERIC)
          123456789123456789.23456789123456789
```

*** String**

[SQL 2023] 4.3 Character strings
[SQL 2023] 6.1 <data type>

```
-- Common
VARCHAR                                        -- base
VARCHAR CHARACTER SET UTF8                     -- with
explicit character set
VARCHAR COLLATE UNICODE                        -- with
explicit collate
VARCHAR CHARACTER SET UT8 COLLATE UNICODE      -- both


-- Character
CHARACTER(10)                      10 characters fixed-
length
CHARACTER                          length 1 if
unspecified
CHAR(10)                           alias

CHARACTER VARYING(10)              10 characters variable
length
CHARACTER VARYING                  any length
CHAR VARYING                       alias
VARCHAR

NATIONAL CHARACTER(10)             international
characters fixed length
NATIONAL CHARACTER                 length 1
NCHAR                              alias

NATIONAL CHARACTER VARYING(10)     variable length
NATIONAL CHARACTER VARYING         any length
NATIONAL CHAR VARYING              alias
NCHAR VARING

-- LARGE OBJECT
CHARACTER LARGE OBJECT
CHAR LARGE OBJECT
CLOB
NATIONAL CHARACTER LARGE OBJECT(10)
NATIONAL CHAR LARGE OBJECT
NCLOB
```

```
-- Large Object sizes
--
-- K     2^10
-- M     2^20
-- G     2^30
-- T     2^40
BLOB(10)
BLOB(1K)          with size
NCLOB(1K)         any *LOB
BLOB(1M)
BLOB(1G)
```

**GOTCHA: meaning of fixed length n in CHARACTER(n)**
https://lets.postgresql.jp/documents/technical/text-processing/1

**GOTCHA**: fixed length n is only **character length n storage**
NOT n bytes storage
NOT length n **validation**.
i.e. does NOT throw error if a given text is shorter or longer.
Instead, the text is cut off or right-padded with spaces.

CHARACTER(n) means:
- **storage**: exactly **n characters**
  NOT n **bytes**

- shorter/longer characters are still accepted
  **Silently** forced to be exactly length n.
  **shorter** characters **right space** padded
  **longer** characters are **cut off**

- **prints** exactly length n

- right-padding is **ignored** in many operations
  e.g.

    'a'::CHAR(10) = 'a'::VARCHAR(100)  TRUE
    'a'::CHAR(10) || 'b'               'ab'

e.g. longer or shorter

```
> SELECT 'a'::CHAR(100);          -- shorter: right-
padded with spaces
                                                  bpchar
________________________________________________________
________________________________________________
 a

> SELECT 'abc'::CHAR(10);
   bpchar
____________
 abc


> SELECT 'abc'::CHAR(1);          -- longer: cutoff
```

```
 bpchar
--------
 a
```

e.g. paddings are ignored

```
> SELECT 'a'::CHAR(10) || 'b';                -- string
concat
 ?column?
----------
 ab

> SELECT 'a'::CHAR(10) = 'a'::VARCHAR(1);      --
comparison
 ?column?
----------
 t
```

*** Binary**
[SQL 2023] 4.4 Binary
[SQL 2023] 6.1 <data type>

```
-- Binary
BINARY(10)                    10 bytes fixed
BINARY

BINARY VARYING(10)            10 bytes max
BINARY VARYING
VARBINARY                     same

-- LARGE OBJECT
BINARY LARGE OBJECT(10)
BINARY LARGE OBJECT
BLOB
```

*** Date and time types**
[SQL 2023] 4.7 Data time
[SQL 2023] 6.1 <data type>

| | |
|---|---|
| **Date** | date only<br>e.g. 2020-01-101 |
| **Time [+timezone]** | time only upto **milliseconds (6 digits)**<br>**Timezone option** = time stored as UTC.<br>e.g. 01:01:1.123456 + 09 (JST)<br>= 16:01:1.123456 + 0 (UTC) |
| **Timestamp [+timezone]** | date + time<br>e.g. 2020-01-01 01:01:1.123456 + 09 |
| **Interval** | range |

e.g.

```
SELECT
  DATE '2025-01-13',
  TIME WITH TIME ZONE '1:2:3.400 JST',
  TIMESTAMP WITH TIME ZONE '2025-01-13 1:2:3.400 JST'
;
```

```
-- Datetime
DATE

-- Time
TIME
TIME(6) WITHOUT TIME ZONE     -- same explicit
TIME(3) ...                   -- millisecond digits e.g.
01:10:20.123
                              -- Max 6
TIME(10) ...                  -- ERROR
TIME WITH TIME ZONE           -- e.g. 01:10:30.123456 +
09
TIME WITHOUT TIME ZONE        -- e.g. 01:10:30.123456


-- Timstamp
TIMESTAMP(5) WITH TIMEZONE    DATE + TIME together
                              e.g. 2000-01-01
01:10:30.123456 + 09
TIMESTAMP WITH TIMEZONE


-- INTERVAL
INTERVAL
  YEAR
  MONTH
  DAY
  HOUR
  MINUTE
  SECOND
  YEAR TO MONTH
  DAY TO HOUR
  DAY TO MINUTE
  HOUR TO MINUTE
  HOUR TO SECOND
  MINUTE TO SECOND
INTERVAL YEAR TO DAY          -- ERROR. Not a supported
range
INTERVAL SECOND(3)            -- upto 3 precision of
seconds e.g. 1.234
INTERVAL DAY(3)               -- ERROR. Must include
SECOND
INTERVAL DAY TO SECOND(3)     -- OK
```

*** Collections (array, multiset, JSON)**
[SQL 2023] 4.12 Collection types

```
INTEGER ARRAY[10]
INTEGER ARRAY             unlimited length

-- [Non standard] PostgreSQL
-- ARRAY is optional
INTEGER[10]
INTEGER[]                 unlimited

INTEGER MULTISET
JSON
```

*** Character Set (aka Character Encoding), Collation**
[SQL 2023] 4.3 Character strings

**Character set aka Character encoding**
e.g. UTF8, UTF16

**Collation**
Method for character **comparison**.
e.g. UNICODE
In SQL 2023 4.3 Character strings section,

'A collation is defined by ISO/IEC 14651:2020 as “a process by which two strings are determined to be in exactly one of the relationships of less than, greater than, or equal to one another'.

```
-- Character set
VARCHAR CHARACTER SET
  ISO8BIT
  GRAPHIC_IRV
  LATIN1
  UTF8
  UTF16
  UTF32

  SQL_CHARACTER
  SQL_TEXT
  SQL_IDENTIFIER


--  COLLATE
VARCHAR COLLATE
  ISO8BIT
  GRAPHIC_IRV
  LATIN1
  UCS_BASIC
  UNICODE
  SQL_CHARACTER
```

```
  SQL_TEXT
  SQL_IDENTIFIER
```

* usage: specify collation of a value
In general, 2 places to specify collation:
a) **value** e.g. 'abc' COLLATION "POSIX"
b) type specification e.g. name VARCHAR COLLATE "POSIX"

```
-- usage: collation of a value
SELECT 'a' COLLATE "POSIX";
SELECT 'a' > 'b' COLLATE "POSIX";

-- usage [Non-standard]: ORDER BY
SELECT * FROM t ORDER BY COLLATE "POSIX";

-- usage: collation of a data type
CREATE TABLE mytable (
  name VARCHAR COLLATE "POSIX"
);
```

*** DOMAIN**
[SQL 2023] 4.14 Domains
[SQL 2023] 11.34 <domain definition>

**Domain**
Custom type for building **domain language**.
In implementation, a **type alias** with e.g. optional constraints.

e.g. natural number

```
-- create type
CREATE DOMAIN nat AS INTEGER CHECK (nat > 0);

-- cast
SELECT CAST (1 AS nat);

-- in table
CREATE TABLE t (v nat);
```

```
CREATE DOMAIN nat AS INTEGER;
CREATE DOMAIN nat AS INTEGER
  -- default value
  DEFAULT 1

  -- constraints
  CHECK (nat > 0)
  CONSTRAINT c CHECK (nat > 0)

  -- collation
  COLLATE "POSXI"
```

*** TYPE (composite type)**
[SQL 2023] 4.9 User defined types
[SQL 2023] 11.51 <user-defined type>

**Type**
Custom user defined type, including a **composite type**.

e.g.

```
CREATE TYPE mypoint AS (x INT, y INT);

SELECT CAST((1, 2) AS mypoint);
  row
=======
 (1,2)

                                        ^
SELECT (CAST((1, 2) AS mypoint)).x;

 x
===
 1
```

```
-- composite type
CREATE TYPE mytype
  AS (
    x INT,
    y VARCHAR COLLATE "A"
 )

-- collection type
CREATE TYPE
 AS INT ARRAY
 AS MULTISET

 ...


-- composite type value
CAST (ROW(1, 'a') AS mytype)  -- with explicit CAST
'(1,"a")'                     -- from string
ROW(1,'a')                    -- from ROW
(1, 'a')                      -- same
(1)                           -- ERROR
ROW(1)


-- composite type access
-- v is a type (x, y)
(v).x                    -- OK
v.x                      -- ERROR. v is interpreted as a
table
(mytable.v).x            -- OK
```

```
-- unwrap
(v).*
(v).w.*                    -- nested
v.*                        -- ERROR
```

```
-- e.g.
SELECT * FROM t;
SELECT (t).* FROM t;                    -- same

SELECT CAST(ROW(1,'a') AS v);           -- as a row
   row
─────────
 (1,'a')

SELECT (CAST(ROW(1,'a') AS v).*;

 x | y
───┼───
 1 | 'a'
```

```
-- [PostgreSQL]
(1,'a')::mytype            -- same as CAST
((1,2)::mytype).x          -- access
```

*** Data type Conversions aka CAST**
[SQL 2023] 4.13 Data conversions
[SQL 2023] 6.13 <cast specification>

Data conversions may be **implicit** or **explicit** via **CAST**
e.g.

```
CREATE TABLE t (x int);

-- Implicit conversions
INSERT INTO t (x) VALUES  -- in INSERT
   (1),                   -- no conversion
   ('1')                  -- with implicit conversion
;
UPDATE t set x = '-2';    -- in UPDATE


-- Explicit conversion
SELECT CAST('1' AS INT);
```

Possible conversions defined in 6.13 <cast specification>
**SD** = source data type
**TD** = Target data type

7) 09 14 If the <cast operand> is a <value expression>, then the valid combinations
<cast specification> are given by the following table. "Y" indicates that the combina
valid without restriction; "M" indicates that the combination is valid subject to
in this Subclause being satisfied; and "N" indicates that the combination is not v
of a cell are unspecified, that also indicates that the combination is not valid.

| *SD* | *TD* | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | EN | AN | DF | C | D | T | TS | YM | DT | BO | UDT | B | RT | CT | R |
| EN | Y | Y | Y | Y | N | N | N | M | M | N | M | N | M | N | N |
| AN | Y | Y | Y | Y | N | N | N | N | N | N | M | N | M | N | N |
| DF | Y | Y | Y | Y | N | N | N | N | N | N | M | N | M | N | N |
| C | Y | Y | Y | Y | Y | Y | Y | Y | Y | Y | M | N | M | N | N |
| D | N | N | N | Y | Y | N | Y | N | N | N | M | N | M | N | N |
| T | N | N | N | Y | N | Y | Y | N | N | N | M | N | M | N | N |
| TS | N | N | N | Y | Y | Y | Y | N | N | N | M | N | M | N | N |
| YM | M | N | N | Y | N | N | N | Y | N | N | M | N | M | N | N |
| DT | M | N | N | Y | N | N | N | N | Y | N | M | N | M | N | N |
| BO | N | N | N | Y | N | N | N | N | N | Y | M | N | M | N | N |
| UDT | M | M | M | M | M | M | M | M | M | M | M | M | M | M | N |
| B | N | N | N | N | N | N | N | N | N | N | M | Y | M | N | N |
| RT | M | M | M | M | M | M | M | M | M | M | M | M | M | N | N |
| CT | N | N | N | N | N | N | N | N | N | N | M | N | N | M | N |
| RW | N | N | N | N | N | N | N | N | N | N | N | N | N | N | M |
| JS | N | N | N | N | N | N | N | N | N | N | N | N | N | N | N |

Where:
EN = Exact Numeric
AN = Approximate Numeric
DF - Decimal Floating-Point
C = Character (Fixed- or Variable-Length, or Character Large C
D = Date
T = Time
TS = Timestamp
YM = Year-Month Interval
DT = Day-Time Interval
BO = Boolean
UDT = User-Defined Type
B = Binary (Fixed- or Variable-Length or Binary Large Object)
RT = Reference type
CT = Collection type
RW = Row type
JS = JSON type

```
CREATE DOMAIN myint AS INT;
CREATE DOMAIN mystring AS VARCHAR;


-- to BOOLEAN
CAST(1 AS BOOLEAN)        -- TRUE
CAST(0 AS BOOLEAN)        -- FALSE
CAST(1.0 AS BOOLEAN)      -- ERROR
CAST('TRUE' AS BOOLEAN)   -- TRUE
CAST('true' AS BOOLEAN)   -- same
CAST('t' AS BOOLEAN)      -- same
CAST('1' AS BOOLEAN)      -- same
CAST('a' AS BOOLEAN)          -- ERROR
CAST(DATE ... AS BOOLEAN)     -- ERROR
CAST(ROW(1, 'a'), AS BOOLEAN) -- ERROR


-- to numeric type
CAST(TRUE AS INT)         -- 1
CAST(TRUE AS NUMERIC)     -- ERROR
CAST(TRUE AS REAL)        -- ERROR
CAST('1' AS INT)
CAST('1.1' AS INT)        -- ERROR
CAST('a' AS INT)          -- ERROR
CAST('123456789' AS REAL)          -- 12.34568
CAST('12.345689' AS NUMERIC(5,3)) -- 12.346
```

```
CAST('123' AS NUMERIC(5,3))       -- ERROR. Does fit in
XX.YYY
CAST(DATE ..., AS INT)        -- ERROR
CAST(ROW(...), AS INT)        -- ERROR


-- to String type: always works
CAST(12345.6789 AS VARCHAR)           -- '12345.6789'
CAST(TRUE AS VARCHAR)                 -- 'true'
CAST(ROW(1, 'a'), AS VARCHAR)         -- '(1, "a")'
-- Trailing cutoff
CAST(TRUE AS VARCHAR(2))              -- 'tr'
CAST(12345.6789 AS VARCHAR(2))        -- '12'
CAST(DATE '2024-01-02', AS VARCHAR(2)) -- '20'
CAST(ROW(1, 'a'), AS VARCHAR(2))      -- '(1'
```

==========
======

**== Predicates ==**

[SQL 2023] 8 Predicates

*** BASIC**

```
-- NULL
x IS NULL
x IS NOT NULL



-- BETWEEN
a BETWEEN x AND y                 -- same as x <= a ABD
a <= y
a BETWEEN ASYMMETRIC y AND x      -- same
a BETWEEN SYMMETRIC x AND y       -- symmetric i.e.
order of x, y doesn't matter
a BETWEEN SYMMETRIC y AND x       -- same


-- Patterns
x LIKE '%'
x NOT LIKE ...
x LIKE 'a#%' ESCAPE '#'    -- use # to escape special
characters
x SIMILAR TO '[a-z]%'
x NOT SIMILAR TO ...


x IS NFC NORMALIZED
x IS NFK NORMALIZED


-- Date time comparison
(d1, d2)                  -- TRUE if
```

```
  OVERLAPS (d3, d4)
(d1, d2)                  -- GOTCHA.
  OVERLAPS (d2, d4)
```

```
-- e.g.
(DATE '2024-01-01', DATE '2024-01-03')             --
TRUE
  OVERLAPS (DATE '2024-01-02', DATE '2024-01-04')

(DATE '2024-01-01', DATE '2024-01-03')             --
GOTCHA. FALSE if same date
  OVERLAPS (DATE '2024-01-03', ...)

-- (DATE, INTERVAL):
(DATE '2024-01-01', INTERVAL '1 DAY 1 HOUR')       --
TRUE  (2024-01-01 0:0:0, 2024-02-01 01:0:0) overlaps
with (2024-01-02 0:0:0, ...)
  OVERLAPS (DATE '2024-01-02', DATE '2024-01-04')
(DATE '2024-01-01', INTERVAL '1 DAY')              --
FALSE (2024-01-01 0:0:0, 2024-02-01 0:0:0) NOT overlaps
with (2024-01-02 0:0:0, ...)
  OVERLAPS (DATE '2024-01-02', DATE '2024-01-04')

-- (DATE, TIME)
select (DATE '2024-01-01', TIME '1:0:1')           --
TRUE (2024-01-01 0:0:0, 2024-02-01 1:0:1) overlaps with
(2024-01-02 1:0:0, ...)
  OVERLAPS (TIMESTAMP '2024-01-01 1:0:0', ...)
select (DATE '2024-01-01', TIME '1:0:0')           --
TRUE
  OVERLAPS (TIMESTAMP '2024-01-01 1:0:0', ...)

-- (TIME, TIME)
select (TIME '0:0:0', TIME '0:1:1')                --
TRUE (0:0:0, 0:1:1) overlaps with (0:1:0, ...)
  OVERLAPS (TIME '0:1:0', TIME '0:2:0')
```

```
---- Query predicates
EXISTS (SELECT ...)          -- TRUE if non-empty


-- IN
--
-- Either:
-- a) special value list,
-- b) subquery
x IN (1, 2, 3)               -- value list
x IN ROW(1, 2, 3)            -- ERROR. Value list is
NOT the same as a row
(x, y) IN (SELECT ...)       -- subquery
```

```
x NOT IN ...                    -- NOT


-- ALL/SOME:
--
(1, 'a') = ALL (SELECT ...)           -- TRUE if all are
(1, 'a')
1 = ALL (SELECT ...)                  -- single

-- operators
... = ALL (SELECT ...)
... <> ALL (SELECT ...)
... < ALL (SELECT ...)
... <= ALL (SELECT ...)
... > ALL (SELECT ...)
... => ALL (SELECT ...)
... IS DISTINCT FROM ALL (...)        -- ERROR. IS
DISTINCT FROM is not an operator
                                      -- ref: https://
dba.stackexchange.com/questions/60342/can-is-distinct-
from-be-combined-with-any-or-all-somehow
... SOME (SELECT ...)
... ANY (SELECT ...)


-- UNIQUE:
UNIQUE (SELECT ...)
UNIQUE NULLS DISTINCT (SELECT ...)
UNIQUE NULLS NOT DISTINCT (SELECT ...)

-- MATCH:
ROW(1, 'a') MATCH (SELECT ...)
... MATCH UNIQUE SIMPLE (SELECT ...)
... MATCH UNIQUE (SELECT ...)
... MATCH SIMPLE (SELECT ...)
... MATCH PARTIAL (SELECT ...)
... MATCH FULL (SELECT ...)


-- multiset
MULTISET[...] IS A SET
MULTISET[...] IS NOT A SET
x IS MEMBER OF MULTISET[...]
x IS NOT MEMBER OF MULTISET[...]
s IS SUBMULTISET OF MULTISET[...]
s IS NOT SUBMULTISET OF MULTISET[...]
```

*** comparison =, <>, <=, >=**

**Equality = <>**
**Order < <= > >=**
Lexicographic order for rows

e.g.

```
-- boolean
TRUE > TRUE               -- FALSE

-- numeric
2 > 1

-- string
'b' > 'a'                 -- TRUE

-- date time
DATE '2025-01-01'         -- TRUE
  > DATE '2000-01-01'

-- row
ROW(1, 'b') > ROW(1, 'a')     -- TRUE
ROW(2, 'a') > ROW(1, 'a')     -- TRUE
```

```
-- Syntax
--
x = y
x <> y
x < y
x <= y
x > y
x >= y
```

*** IS NULL**
[SQL 2023] 8.8 <null predicate>

**IS NULL**
Determines whether a value is **NULL**.
For composite types

| | |
|---|---|
| ROW(...) NOT NULL | TRUE only if **all columns** are NULL |
| ROW(...) IS NOT NULL | TRUE only if **all columns** are non-NULL |

e.g

```
1 IS NULL                 -- F
NULL IS NULL              -- T
(NULL) IS NULL            -- T
-- row/composite
ROW(NULL) IS NULL         -- T 1 column row, all NULL
ROW(NULL, NULL) IS NULL   -- T 2 column row, all NULL
ROW(1, NULL) IS NULL      -- F 1 column NULL
ROW(1, 2) IS NOT NULL     -- F all columns not NULL


1 IS NOT NULL                 -- T
NULL IS NOT NULL              -- F
(NULL) IS NOT NULL            -- F
```

```
-- row/composite
ROW(NULL) IS NOT NULL          -- F
ROW(NULL, NULL) IS NOT NULL    -- F
ROW(1, NULL) IS NOT NULL       -- F
ROW(1, 2) IS NOT NULL          -- T
```

**GOTCHA**: a row may be neither IS NULL nor IS NOT NULL
e.g.

```
ROW(1, NULL) IS NULL           -- F
ROW(1, NULL) IS NOT NULL       -- F
```

```
-- Basic syntax
x IS NULL
x IS NOT NULL
```

*** IS DISTINCT FROM**
[SQL 2023] 8.15 <distinct predicate>

**IS NOT DISTINCT vs equality =, <>**
IS DISTINCT FROM is like equality but **never NULL**

```
-- equality =
1 = 1                              -- T
1 = NULL                           -- NULL
NULL = NULL                        -- NULL
ROW(NULL) = NULL                   -- NULL
ROW(NULL) = ROW(NULL)              -- NULL
ROW(1) = ROW(NULL)                 -- NULL
ROW(1, NULL) = ROW(1, NULL)        -- NULL


-- IS NOT DISTINCT FROM
1 IS NOT DISTINCT FROM 1                            -- T
1 IS NOT DISTINCT FROM NULL                         -- F
NULL IS NOT DISTINCT FROM  NULL                     -- T
ROW(NULL) IS NOT DISTINCT FROM  NULL                -- F
ROW(NULL) IS NOT DISTINCT FROM ROW(NULL)            -- T
ROW(1) IS NOT DISTINCT FROM ROW(NULL)               -- F
ROW(1, NULL) IS NOT DISTINCT FROM ROW(1, NULL)      -- T
```

```
x IS DISTINCT FROM y
x IS NOT DISTINCT FROM y      -- NOT
1 IS DISTINCT FROM 'a'        -- ERROR. Has to be same
type
1 IS DISTINCT FROM NULL       -- OK. Can compare against
NULL e.g. T
```

*** LIKE, SIMILAR TO**
[SQL 2023] 4.3.3 Operations involving character strings
[SQL 2023] 8.5 <like predicate>
[SQL 2023] 8.6 <similar predicate>

**SIMILAR TO**
Regular expression-like syntax

e.g.

```
-- LIKE
SELECT 'abc' LIKE 'a';             -- FALSE
SELECT 'abc' LIKE 'abc';           -- TRUE
SELECT 'abc' LIKE 'a%';            -- TRUE. Wildcard.
SELECT 'abc' LIKE '_bc';           -- TRUE. Single
wildcard
SELECT 'ab%' LIKE 'ab[%]';
SELECT 'ab%' LIKE 'ab@%' ESCAPE '@';   -- TRUE. Use @ to
escape

-- SIMILAR TO
SELECT 'abc' SIMILAR TO 'a';       -- FALSE
SELECT 'abc' SIMILAR TO 'abc';     -- TRUE
SELECT 'abc' SIMILAR TO 'a%';      -- TRUE. Wildcard.
SELECT 'abc' SIMILAR TO '_bc';     -- TRUE. Single
wildcard
SELECT 'abc' SIMILAR TO '[a-z]+'; -- TRUE. Range
SELECT '[ab]' SIMILAR TO '@[ab@]' ESCAPE '@';       --
TRUE. use @ to escape []
```

# LIKE syntax

| | |
|---|---|
| % | all strings e.g. 'abc' 'def' |
| %abc | e.g. '123abc' |
| abc% | e.g. 'abc123' |
| abc_ | single wildcard<br>e.g. OK 'abc1'<br>NO 'abc12' |
| [%] | escape '%' |
| [_] | escape '_' |
| [a-zA-Z0-9_] | characters |

# SIMILAR TO

| | |
|---|---|
| % | same as LIKE |
| _ | |
| a\|b | OR |
| a* | |
| a+ | |
| a? | 0 or 1 |
| a{2} | exactly 2 |
| a{2,} | 2 or more |
| a{2,5} | |
| (a\|b)+ | group with () |

[a-zA-Z0-9]

```
-- Patterns
--
x LIKE 1             -- exact match
x LIKE '1%'          -- Wildcard. i.e. .* in regex
x LIKE '1_'          -- Single character wildcard.
i.e. . in regex
x NOT LIKE '1%'
```

==========

## == Expressions ==

[SQL 2023] 6.3 <value expression primary>

*** Basic**

```
-- CASE
CASE WHEN x > 0 THEN 'A'
     WHEN x < 10 THEN 'B'
     ELSE 'C'
END

CASE x WHEN 0 THEN 'A'        -- same as WHEN x = 0
       WHEN 1 THEN 'B'
       ELSE 'C'
END

-- shorthands
--
-- x || y
COALESCE(x, y)                                  --
returns first non-NULL
CASE WHEN x IS NOT NULL THEN x ELSE y END       -- same

-- inverse of COALESCE
--   x == null ? x | y    COALESCE
--   x == y ? null | x    NULLIF
NULLIF(x, y)                                    --
returns x
CASE WHEN x = y THEN NULL ELSE x END            -- same



-- subquery
(SELECT 1)


-- min/max
```

```
GREATEST(1,2,3,4)
GREATEST('a','b','c')     -- OK with any comparable value
LEAST(1,2,3,4)
```

*** Boolean**

[SQL 2023] 6.32 <boolean value expression>
[SQL 2023] 6.33 <value function>

```
-- literals
TRUE
true
FALSE
false

-- operator
x AND y
x && y
x OR y
x || y
NOT x
! x


-- Boolean with NULL aka 3 valued logic
--
-- Wiki: https://en.wikipedia.org/wiki/
SQL_syntax#Null_or_three-valued_logic_(3VL)
TRUE AND NULL         -- NULL
FALSE AND NULL        -- FALSE
NULL AND NULL         -- NULL
TRUE OR NULL          -- TRUE
FALSE OR NULL         -- NULL
NULL OR NULL          -- NULL
NOT NULL              -- NULL
```

*** Numeric**

[SQL 2023] 6.30 <numeric value expression>
[SQL 2023] 6.31 <numeric value function>

```
-- literal
1                -- integer
1.1              -- real



-- operators
x + y
x - y
x / y
x % y
```

```
-- functions
MOD(5, 3)        -- 5 % 2 = 2
ABS(1)
FLOOR(1.1)
CEIL(1.1)
CEILING(1.1)

-- SIN, COS, TAN
-- COSTH, TANH
-- ASIN, ACOS, ATAN
SIN(1)
COS(1)
TAN(1)

LOG10(1)
LN(1)
EXP(3)               -- e^3
POWER(2, 3)          -- 2^3
```

*** String/binary**
[SQL 2023] 6.32 <string value expression>
[SQL 2023] 6.33 <string value function>

**String**
1-indexed

```
-- literal string
'a'
"a"              -- GOTCHA: "" is a name (e.g. table
name, column name), NOT a string
                 -- e.g. SELECT "column A" FROM "my
table"
''''             -- escape '
'[''a'']'        -- e.g. ['a']

-- collation
'a' COLLATE "POSIX"       -- specify collation
'a'


-- operators
'a' || 'b'                   -- concat 'ab'

-- basic
CHARACTER_LENGTH('abc')      -- 3
CHAR_LENGTH('abc')
UPPER(x)
LOWER(x)

-- index
POSITION('bc' IN 'abc')      -- 2 1-indexed position
POSITION('x' IN 'abc')       -- 0 if not in string
```

```
-- access
SUBSTRING(x FROM 2)           -- x[2]x[3]...
SUBSTRING(x FROM 1)           -- full string
SUBSTRING(x FROM 2 FOR 3)     -- x[2]x[3]x[4]
SUBSTRING(x FOR 2)            -- x[1]x[2]
SUBSTRING(x FROM -1)          -- OK. From start
SUBSTRING(x FOR 10000)        -- OK. Until end


-- replace
OVERLAY(x PLACING 'abc' FROM 2)               --
x[2]='a' x[3]='b' x[4]='c'
OVERLAY(x PLACING 'abc' FROM 2 FOR 8)         -- fill
rest with empty i.e. ' ' =  [5]=x[6] ...
```

```
> OVERLAY('abcdefg' PLACING 'uv' FROM 2)
'auvcdefg'

> OVERLAY('abcdefg' PLACING 'uv' FROM 2 FOR 2)     --
same
'auvcdefg'

> OVERLAY('abcdefg' PLACING 'uv' FROM 2 FOR 0)     --
same if < length
'auvcdefg'

> OVERLAY('abcdefg' PLACING 'uv' FROM 2 FOR 10)    --
fill with empty if > length
'auvc    '
```

```
-- TRIM, BTRIM/LTRIM/RTRIM: trim characters
TRIM(x)                       -- trim white spaces
TRIM(LEADING 'ab' FROM x)     -- all leading occurrences
of 'a' and 'b'
TRIM(LEADING FROM x)          -- whitespace
TRIM(LEADING ' ' FROM x)      -- same
TRIM('a' FROM x)              -- same as BOTH
TRIM(TRAILING ...)
TRIM(BOTH ...)
```

```
> TRIM('   abc   ')
'abc'

> TRIM('ab' FROM 'aabbcabdbbb')
'caad'

> TRIM(LEADING 'ab' FROM 'aabbcabdbbb')
'cabdbbb'
```

```
LTRIM(x, 'a')        -- same as TRIM(LEADING 'a' FROM x)
```

```
LTRIM(x)              -- LTRIM(x, ' ')
RTRIM(...)            -- TRIM(TRAILING ...)
BTRIM(...)            -- TRIM(BOTH ...)


-- LPAD, RPAD: pad/cutoff to fit size
LPAD(x, 5)
LPAD(x, 5, 'abc')
LPAD(x, 5, ' ')       -- default
RPAD(...)
```

```
> LPAD('abc', 5)          -- basic
'  abc'

> LPAD('abc', 2)          -- cutoff
'ab'

> LPAD('abc', 0)          -- length 0
''

> LPAD('abc', -1)         -- same
''

> LPAD('abc', 5, 'defgh')     -- partial
'deabc'

> LPAD('abc', 10, 'def')      -- repeat
'defdefdabc'
```

```
-- Unicode normalization
NORMALIZE(x, NFC)
NORMALIZE(x, NFD)
NORMALIZE(x, NFKC)
NORMALIZE(x, NFKD)



-- misc
CONVERT(x USING UTF8)
TRANSLATE(...)
```

*** Row (type)**
[SQL 2023] 4.10 Row type
[SQL 2023] 7.1 <row value constructor>

**Row type**
**Table row type**
A row type is a **composite** type.
Each value is a **field**.
A **table row** is a type of row type

```
ROW(1, 'a')      -- explicit
(1, 'a')         -- same implicit

ROW(1)           -- explicit with 1 field
1                -- same implicit
(1)              -- BAD. Not a row but parenthesized 1

(SELECT ...)     -- row from subquery
                 -- must have at least 1 result
```

*** Binary**

[SQL 2023] 6.32 <string value expression>
[SQL 2023] 6.33 <string value function>

```
-- literal binary
'a'              -- binary  \x61 = 97
'\141'           -- octet   \x61 = 97
'\x61'
'\x61626364'     -- 'abcd'


-- binary functions
OCTET_LENGTH(...)
POSITION('a' in '\x61626364')     -- 1
POSITION('b' in '\x61626364')     -- 2
```

*** Array**

[SQL 2023] 6.47 <array value expression>
[SQL 2023] 6.48 <array value function>

**usage**: map/filter/sort array
Use **ARRAY** + **UNNEST**

```
SELECT ARRAY(
  SELECT x * 2                              -- map
  FROM UNNEST(ARRAY[1, 2, 3]) t(x)
  WHERE x > 1                               -- filter
  ORDER BY x DESC                           -- sort
) a;

   a
_______
 {4,6}
```

**usage**: check if array contains element
Use ANY

```
SELECT 1=ANY(ARRAY[1,2,3])        -- TRUE
```

```
SELECT 5=ALL(ARRAY[1,2,3])        -- FALSE

SELECT 1=ALL(ARRAY[1,1, 1])       -- TRUE
```

```
-- Literal
ARRAY[1,2,3]
ARRAY (VALUE (1), (2))        -- by subquery [1,2]


-- GOTCHA: empty array
ARRAY[]                       -- WARNING. May not be
able to determine type
CAST(ARRAY[] AS INT ARRAY)    -- Cast as INT array
-- [Non-standard PostgreSQL]
ARRAY[]::INT ARRAY
ARRAY[]::INT[]


-- UNNEST: dual of subquery
UNNEST(array[1,2])        -- array to rows
```

```
-- query
> VALUE(1), (2);
 column
________
      1
      2

-- as array
> SELECT ARRAY(VALUE(1), (2));
 column
___________
 {1,2}

-- back to query
> SELECT * FROM UNNEST(ARRAY(VALUE(1), (2)));
> SELECT UNNEST(ARRAY(VALUE(1), (2)));             --
same
unnest
________
      1
      2
(1 row)
```

```
UNNEST(                       -- multiple
  ARRAY[1,2],
  ARRAY[3,4,5]
) _(a,b)
UNNEST(ARRAY[1,2])            -- i = 1-based index
```

```
  WITH ORDINALITY _(a,i)
```

```
-- multiple
> SELECT * FROM UNNEST(ARRAY[1,2], ARRAY[3,4,5]) _(a, b)
  a   | b
------+---
    1 | 3
    2 | 4
 NULL | 5


> SELECT * FROM UNNEST(ARRAY[3,4]) WITH ORDINALITY
_(a,i) ;
 a | i
---+---
 3 | 1
 4 | 2
(2 rows)
```

```
-- Array: 1-indexed
x[1]                  -- first element
x[0]                  -- NULL
x[NULL]               -- NULL
x[10000]              - NULL if out of range
x[0] IS NULL          -- True

-- [NON-standard: PostgreSQL]
x[1:10]                    -- range

-- operations
ARRAY[1,2] || ARRAY[3,4]      -- concat [1,2,3,4]
NULL || ARRAY[1, 2]           -- with NULL [1,2]

-- functions
CARDINALITY(x)
ARRAY_MAX_CARDINALITY(x)

-- [Non-standard: e.g. PostgreSQL]
array_length(x)


-- [NON-standard: PostgreSQL]
1=ANY(ARRAY[1,2,3])       -- array contains 1
1=SOME(ARRAY[1,2,3])      -- same
1=ALL(ARRAY[1,1,1])       -- all are 1
```

Multiset

```
-- Literal
MULTISET[1,2,3]
```

```
... etc
```

*** Date and time**

[SQL 2023] 6.42 <datetime value expression>
[SQL 2023] 6.43 <datetime value function>

# date
2024-01-13 ISO 8601 January 02, 2024
20240113 ISO 8601
240113 ISO 8601
January 13, 2024
Jan 13, 2024
1/13/2024
1/13/24
2024-Jan-02
24-01-02
Jan-02-2024
January 13, 2024 BC

# time
01:22:33.456789 ISO 8601
01:22:33 ISO 8601
01:22 ISO 8601
010203.456789 ISO 8601
01:22 AM
01:22 PM

# timestamp
2024-01-01 1:2:3.456789
2024-01-01 1:2:3.456789 JST timezone
2024-01-01 1:2:3.456789 Japan full timezone name
2024-01-01 1:2:3.456789 +9 offset

# interval
2 years
1 year
2 months
1 month
2 years 2 months 2 weeks 2 days 2 hours 2 minutes 2.123456 seconds
...

```
---- Literals
-- See 5.3 <literal>
DATE '2024-01-13'
TIME '1:2:3.4567'
TIMESTAMP WITH TIME ZONE '2024-01-01 1:2:3.4567 JST'
    -- 2023-12-31 16:02:03.4567+00 (UTC)
TIMESTAMP WITHOUT TIME ZONE '2024-01-01 1:2:3.4567'
TIMESTAMP '2024-01-01 1:2:3.4567'
    -- same
DATE '2024-01-01 1:2:3.4567 JST'
    -- OK '2024-01-01'
TIME '2024-01-01 1:2:3.4567 JST'
    -- OK '1:2:3.4567'

-- Interval
INTERVAL '2 years 2 months 2 weeks'
```

```
-- timezone map
-- https://www.postgresql.org/docs/current/functions-
datetime.html#FUNCTIONS-DATETIME-ZONECONVERT
TIME '1:22' AT LOCAL                            -- local
timezone e.g. '1:22:00+09' if JST
TIME '1:22' AT TIME ZONE 'JST'                  --
Timezone by name
TIME '1:22' AT TIME ZONE '+9'                   -- same
TIME '2024-1-13 1:22' AT TIME ZONE 'JST'        -- OK.
Timestamp
TIME '2024-1-13' AT TIME ZONE 'JST'             -- ERROR.
No time


-- functions
CURRENT_TIMESTAMP                  -- current timestamp
with timezone e.g. '2024-1-13 1:22:33.123456 + JST'
CURRENT_TIMESTAMP(2)               -- upto 2 milisecond
digits e.g. '2024-1-13 1:22:33.12 + JST'
LOCALTIMESAMP                      -- CURRENT_TIMESTAMP
without timezone
LOCALTIMESAMP(2)
CURRENT_DATE                       -- date only e.g.
'2024-1-13'
CURRENT_TIME                       -- time only, with
timezone e.g. '1:22:33.123456 + JST'
CURRENT_TIME(2)
LOCALTIME                          -- CURRENT_TIME
without timezone
LOCALTIME(2)
```

*** VALUES list (adhoc TABLE) [PostgreSQL only?]**
VALUES: https://www.postgresql.org/docs/current/queries-values.html

**VALUES**
Creates an **adhoc table**
By default, column names are **column1, column2**, ...

e.g.

```
> VALUES (1, 'a'), (2, 'b');
 column1 | column2
─────────┼─────────
       1 | a
       2 | b


> SELECT column1 AS x, column2 AS y
                     -- default column names
FROM (VALUES (1, 'a'), (2, 'b'));
 x | y
───┼───
```

```
 1 | a
 2 | b

> SELECT *
FROM (VALUES (1, 'a'), (2, 'b')) AS _(x,
y);          -- same with alias
 x | y
---+---
 1 | a
 2 | b
```

*** UNNEST**
[SQL 2023] 7.6 <table reference>

Opposite of **aggregation**
i.e.

| | | | |
|---|---|---|---|
| **aggregation** | rows | -> | array value |
| **unnest** | array value | -> | rows |

e.g.

```
SELECT * FROM UNNEST(ARRAY[1, 2, 3]);

-- with alias
SELECT x FROM UNNEST(ARRAY[1, 2, 3]) t(x);
```

*** Aggregate function**
[SQL 2023] 10.9 <aggregate function>

**Aggregate function**
[SQL 2023] 10.9 <aggregation function>

**usage**
If a SELECT item contains an aggregate function, the other items must either be
a) another **aggregate**, or
b) **GROUP BY** key

e.g.

```
SELECT array_agg(a) FROM t;                          -- OK

SELECT array_agg(a), array_agg(b) FROM t;            -- OK
both aggregates

SELECT array_agg(a), b FROM t;                       --
ERROR

SELECT array_agg(a), b FROM t GROUP BY b;            -- OK
```

```
aggregate and group key
```

NULLs are handled as expected:
e.g. included for ARRAY_AGG
e.g. ignored for MAX, SUM

```
CREATE VIEW v (x,y) AS
  VALUES (1,NULL), (NULL,NULL), (2,NULL);

SELECT
  ARRAY_AGG(x),           -- included [1, NULL, 2]
  SUM(x),                 -- 2
  MAX(x),                 -- 3

  -- all NULLs
  ARRAY_AGG(y),           -- included [NULL, NULL, NULL]
  SUM(y),                 -- NULL
  MAX(y)                  -- NULL
FROM v;
```

```
-- General
ARRAY_AGG(x)
ARRAY_AGG(x + 1)                      -- expression
ARRAY_AGG(DISTINCT x)                 -- DISTINCT values
ARRAY_AGG(x ORDER BY y)               --
ARRAY_AGG(x) FILTER (WHERE x > 0)
ARRAY_AGG(ROW(x, y))                  -- aggregate
composite


-- Count
COUNT(*)
COUNT(*) FILTER (WHERE X > 0)


-- Unary function
AVG(X)
AVG(ALL X)
AVG(DISTINCT X)


-- handling of NULL:
-- - NULLs are excluded
-- - result is NULL if all values are NULL
AVG(X)    -- X=NULL,1,2  AVG=1.5
AVG(X)    -- X=NULL,NULL AVG=NULL


MAX(x)
MIN(x)
AVG(X)
```

```
SUM(x)
STDDEV_POP(x)
STDDEV_SAMP(x)
VAR_POP(x)
VAR_SAMP(x)

ANY(x)
SOME(x)
BOOL_OR(x)   # PostgreSQL
BOOL_AND(x)

COLLECT(x)
FUSION(x)
INTERSECT(x)


-- Binary function
COVAR_POP(x, y)
COVAR_SAMP(x, y)
CORR(x, y)
...

-- array
ARRAY_AGG(x)
ARRAY_AGG(x ORDER BY y ASC)
```